

Lesson 12 — K-Means Clustering

Now we move from Supervised Learning → Unsupervised Learning.

This is an important change.

Up to Lesson 11, we mostly had:

```
X (features) + y (target)
      ↓
    Model
      ↓
  Prediction
```

With K-Means, we usually don't have a target y .

Instead, we give the algorithm data and ask:

“Can you find natural groups in this data?”

12.1 What is Unsupervised Learning?

In supervised learning, we have labeled data.

Example:

Hours studied → Pass/Fail

Hours	Result
2	Fail
4	Fail
7	Pass
9	Pass

The model learns:

$X \rightarrow y$

In unsupervised learning:

Hours studied

2 4 7 9

There is no Pass/Fail column.

The algorithm tries to discover structure itself.

```
Data
  ↓
Find patterns/groups
  ↓
Clusters
```

12.2 What is Clustering?

Clustering means grouping similar data points together.

Imagine you have customers:

Customer A → spends ₹500 Customer B → spends ₹700 Customer C → spends ₹10,000
Customer D → spends ₹12,000 Customer E → spends ₹15,000

We might discover:

Cluster 1

Low-spending customers

Cluster 2

High-spending customers

Nobody gave the algorithm these labels.

It discovered the groups based on the data.

12.3 What is K-Means?

K-Means is one of the most popular clustering algorithms.

The K means:

Number of clusters we want.

For example:

$K = 3$

means:

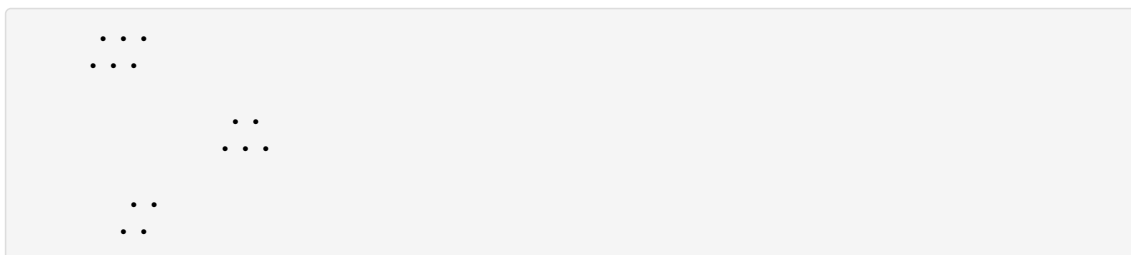
Divide the data into 3 clusters.

So:

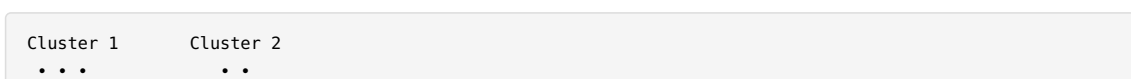
$K = 2 \rightarrow 2$ clusters $K = 3 \rightarrow 3$ clusters $K = 5 \rightarrow 5$ clusters

12.4 Simple Visualization

Imagine these points:



K-Means might discover:



...
Cluster 3
...

The important thing is:

Points within the same cluster should be relatively similar to each other.

12.5 How Does K-Means Work?

This is the most important part of the lesson.

Suppose:

K = 3

K-Means roughly follows these steps:

1. Choose K
↓
2. Select initial centroids
↓
3. Assign each point to nearest centroid
↓
4. Recalculate centroids
↓
5. Reassign points
↓
6. Repeat
↓
7. Stop when clusters stabilize

Let's understand each step.

12.6 Step 1 — Choose K

Suppose we want:

K = 3

So we want:

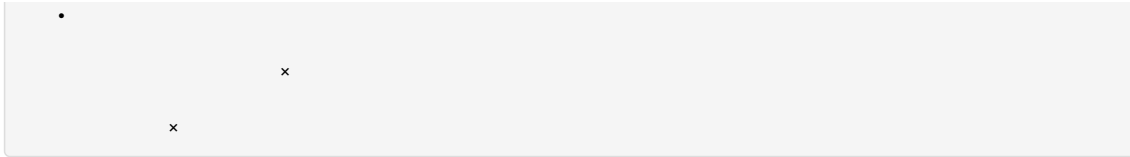
Cluster 1
Cluster 2
Cluster 3

12.7 Step 2 — Choose Centroids

A centroid is the center of a cluster.

Initially, K-Means chooses 3 starting centroids.

•
x



The **x** symbols represent centroids.

12.8 Step 3 — Assign Points

Each data point is assigned to the nearest centroid.

For example:

```
Point A
↓
Nearest centroid
↓
Cluster 1
```

Another:

```
Point B
↓
Nearest centroid
↓
Cluster 2
```

And so on.

12.9 Step 4 — Recalculate Centroids

After assigning points, K-Means calculates a new center for each cluster.

For example:

```
Cluster 1:
. . .
. . .
↓
New centroid
x
```

The centroid moves toward the center of its assigned points.

12.10 Step 5 — Repeat

Now the algorithm assigns the points again based on the new centroids.

Then:

```
Assign
↓
Move centroid
↓
Assign again
```

```
↓  
Move centroid  
↓  
...
```

Eventually the centroids stop moving significantly.

Then K-Means has converged.

The Core Idea

Remember this:

K-Means repeatedly assigns points to the nearest centroid and then moves each centroid to the center of its assigned points.

That's the heart of the algorithm.

12.11 K-Means in Scikit-Learn

Now let's use it.

Import:

```
from sklearn.cluster import KMeans
```

Create the model:

```
model = KMeans(n_clusters=3, random_state=42)
```

Notice:

```
n_clusters=3
```

This is our K.

Then:

```
model.fit(X)
```

And get cluster assignments:

```
labels = model.labels_
```

12.12 Complete Example

Let's create some simple data.

```
import numpy as np  
from sklearn.cluster import KMeans  
  
X = np.array([  
    [1, 2],  
    [1, 3],  
    [2, 2],  
    [8, 8],  
])
```

```

    [9, 8],
    [8, 9],
    [20, 20],
    [21, 20],
    [20, 21]
])

model = KMeans(
    n_clusters=3,
    random_state=42
)

model.fit(X)

print(model.labels_)

```

You might get something conceptually like:

```
[1 1 1 2 2 2 0 0 0]
```

The exact cluster numbers can differ.

For example:

```

Cluster 0
Cluster 1
Cluster 2

```

are just identifiers.

Cluster `0` isn't inherently better than Cluster `1`.

12.13 Important: Cluster Labels Are Not Classes

This is a very important distinction.

In classification:

```

0 = Cat
1 = Dog

```

The labels have a defined meaning.

But with K-Means:

```

0
1
2

```

are simply cluster IDs.

K-Means could return:

```
[0 0 0 2 2 2 1 1 1]
```

instead of:

```
[1 1 1 2 2 2 0 0 0]
```

and the clustering would still be equivalent.

12.14 Getting the Centroids

You can access the cluster centers with:

```
model.cluster_centers_
```

Example:

```
print(model.cluster_centers_)
```

You might get:

```
[[20.33, 20.33],  
 [ 1.33,  2.33],  
 [ 8.33,  8.33]]
```

These represent the centers of the three clusters.

12.15 Predicting a New Point

Suppose we have a new customer:

```
new_point = [[2, 3]]
```

We can ask K-Means which cluster it belongs to:

```
model.predict(new_point)
```

For example:

```
[1]
```

Meaning:

This point is closest to cluster 1's centroid.

12.16 `fit_predict()`

You will often see this:

```
labels = model.fit_predict(X)
```

Instead of:

```
model.fit(X)  
  
labels = model.labels_
```

`fit_predict()` does both:

```
fit()  
+  
predict cluster labels
```

So:

```
labels = model.fit_predict(X)
```

is very convenient for clustering.

12.17 Do We Need Feature Scaling?

Yes, this is important for K-Means.

K-Means uses distance to determine which centroid is closest.

Suppose we have:

Age:	20–60
Income:	10,000–1,000,000

Income has a much larger numerical scale.

It can dominate the distance calculation.

Therefore, we commonly scale features before K-Means.

Example:

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)
```

Then:

```
model = KMeans(
    n_clusters=3,
    random_state=42
)

labels = model.fit_predict(X_scaled)
```

The pipeline becomes:

```
Raw Data
↓
StandardScaler
↓
Scaled Data
↓
KMeans
↓
Clusters
```

This is similar to why scaling was important for KNN and SVM.

12.18 How Do We Choose K?

This is one of the biggest questions:

How do I know whether K should be 2, 3, 4, 5...?

We need a method.

One popular technique is the:

Elbow Method

We calculate a value called inertia for different values of K.

In scikit-learn:

```
model.inertia_
```

You can try:

```
from sklearn.cluster import KMeans

for k in range(1, 11):

    model = KMeans(
        n_clusters=k,
        random_state=42
    )

    model.fit(X_scaled)

    print(k, model.inertia_)
```

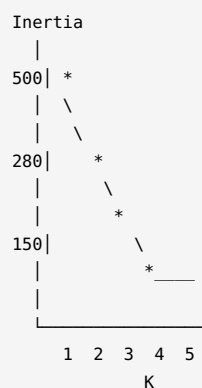
You'll generally see:

K	Inertia
1	500
2	280
3	150
4	120
5	105
6	98
...	

Inertia decreases as K increases.

But eventually the improvement becomes much smaller.

That point looks like an elbow.



If the elbow occurs around:

K = 3

we may choose:

```
n_clusters=3
```

Lesson 12 — What You Need to Know So Far

You should now understand these concepts:

Supervised Learning

```
X + y
↓
Learn relationship
↓
Prediction
```

Unsupervised Learning

```
X
↓
Find hidden structure
↓
Groups/patterns
```

K-Means

```
Choose K
↓
Choose centroids
↓
Assign points
↓
Move centroids
↓
Repeat
↓
Clusters
```

Important Scikit-Learn commands

```
from sklearn.cluster import KMeans
```

```
model = KMeans(n_clusters=3, random_state=42)
```

```
model.fit(X)
```

```
model.fit_predict(X)
```

```
model.labels_
```

```
model.cluster_centers_
```

```
model.inertia_
```

Next Part of Lesson 12

Next we'll go deeper into K-Means practically:

- 12A → Visualizing K-Means clusters
- 12B → StandardScaler + K-Means
- 12C → Elbow Method properly
- 12D → Choosing the optimal K
- 12E → Silhouette Score
- 12F → Complete customer segmentation project
- 12G → K-Means limitations and common mistakes

Then we'll move to Lesson 13 — PCA (Dimensionality Reduction).